Software Re-Engineering

Fall 2026

Lecture 7

Impact Analysis in Software Re-engineering

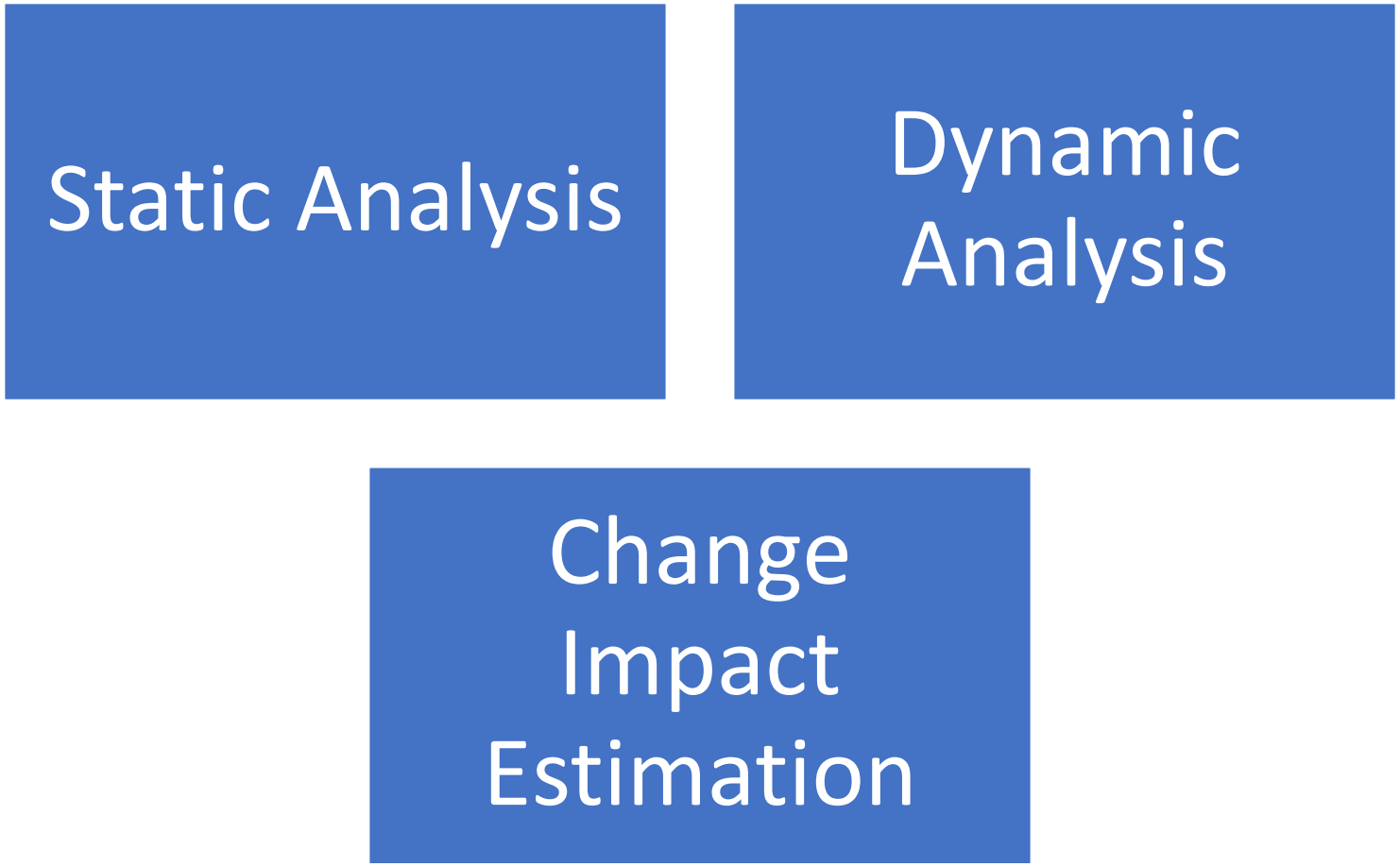
Impact Analysis

- It is the process of identifying and evaluating the consequences of a proposed software change within an existing system.
- It involves assessing how modifications to certain components or code elements can affect other parts of the software.

Importance of Impact Analysis

- Minimizing Risk
  - It ensures that changes do not unintentionally disrupt working parts of the system.
- Resource Allocation
  - Helps in estimating the time and resources required to implement changes.
- Regulatory and Compliance
  - Ensures that changes comply with required standards and do not breach established protocols.
- Change Propagation
  - Understanding how one change might flow through the system.

Techniques for Impact Analysis



Static Analysis

- Examining the source code, architectural models, or documentation without executing the software.
- **Tools and Methods**
  - **Code Linters**: Detect syntactical issues and possible discrepancies in software dependencies.
  - **Control Flow and Data Flow Analysis**
  - Tracks data dependencies and control flows to predict the impact of changes.
  - **Call Graphs**
  - Visual representation of method calls to see which functions might be affected by changes.

Dynamic Analysis

- Analyzing the software during execution to observe actual behavior, interaction, and performance of components.
- **Tools and Methods**
  - **Profiling Tools:** Identify which parts of the system are most frequently executed or likely to be impacted by changes.
  - **Runtime Tracing:** Tracks data propagation during execution to understand interactions.
  - **Test Case Coverage:** Helps evaluate which parts of the system are most at risk of being affected by changes.

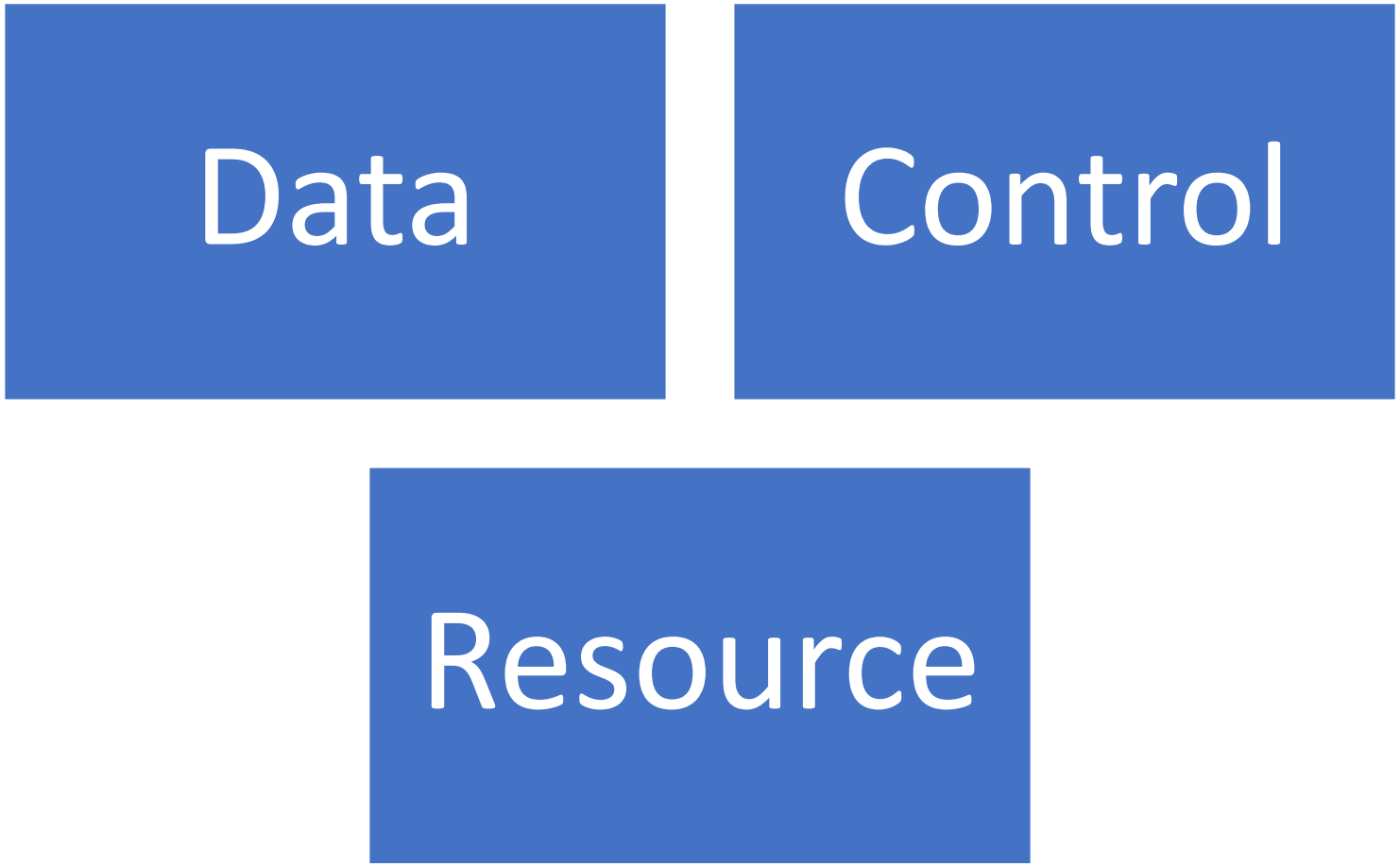
Change Impact Estimation

- Predicting the possible effects of a change on various system components.
- **Tools and Methods**
  - **Mathematical Modeling:** Predicts the behavior of the system based on various change scenarios.
  - **Dependency Matrices:** A matrix that shows relationships between components, allowing us to trace how changes in one part affect others.

Dependency Analysis

- It is the process of identifying and documenting the dependencies between components within a software system.
- Dependencies can be data-driven (where components share or alter data) or control-driven (where components control the flow of execution).

Types of Dependencies



Data Dependencies

- Involves tracking how data flows through the system, such as function calls or variable assignments.
- **Example:** A change in a database schema (e.g., adding a new field) can propagate to various application layers.

Control Dependencies

- Refers to the flow of execution across the system, such as procedure calls or branching statements.
- **Example:** Modifying a decision-making process (e.g., changing conditional logic) can impact the logic of downstream components.

Resource Dependencies

- Involves understanding how different parts of the system rely on shared resources, such as memory, hardware, or network access.
- **Example:** Changing a memory allocation process could affect several processes or services relying on that resource.

Techniques for Dependency Analysis

- **Call Graphs**
  - Visualize which components call each other to track dependencies.
- **Data Flow Diagrams (DFDs)**
  - Shows how data moves between system components.
  - Using tools to extract code structures and relationships between components to identify dependencies.

Change Propagation in Software Re-engineering

- It is the process by which a change in one part of the software system leads to changes in other parts of the system.
- This concept is central to understanding how changes ripple through the system, affecting not just the directly modified components but others that are indirectly influenced.

Direct Propagation

- A direct modification to a function, class, or module that causes a change in other modules it directly interacts with.
  - **Example:** A modification in a core API function might require changes in all client modules that depend on it.

Indirect Propagation

- Changes that are less direct but still have significant downstream effects, often requiring deep analysis to identify.
  - **Example:** Altering a database schema could indirectly cause issues in multiple layers of the application, including the UI, backend, and middleware.

Techniques for Understanding Change Propagation

- **Dependency Graphs:**
  - Graphical representation of the relationships between software components, highlighting potential propagation paths of changes.
- **Sensitivity Analysis:**
  - Analyzing which parts of the system are most sensitive to changes, based on factors like frequency of interaction or criticality of the component.
- **Propagation Modeling:**
  - Using models (e.g., fault propagation models, impact graphs) to simulate how changes in one part of the system can lead to broader consequences.

Metrics for Impact Analysis

- Change Propagation Metrics
- Coupling Metrics
- Cohesion Metrics
- Dependency Metrics
- Complexity Metrics
- Risk Metrics
- Change Volume Metrics
- Traceability Metrics

Change Propagation Metrics

- These metrics help to quantify how changes in one part of the system will affect other parts.
- **Change Propagation Index**
  - Measures how far-reaching or extensive a change might be across the system. A higher CPI indicates more significant impact.
- **Propagation Depth**
  - The number of layers or components that are affected by a change. A deeper propagation depth means more components are impacted.
- **Change Influence Metric**
  - Measures the percentage of system components that are likely to be affected by a change in a given component. This metric is used to evaluate the "influence" of one part of the system over the entire system.

Coupling Metrics

- Measures the degree of dependency or interaction between components. Highly coupled components are more likely to have far-reaching impacts when modified.
- **Coupling Between Objects**
  - Measures how many other classes or modules a particular class interacts with. A higher CBO indicates that changes to that class will likely impact many other classes.
- **Fan-in/Fan-out**
  - Fan-in refers to the number of modules that depend on a particular module.
  - Fan-out refers to the number of modules that a particular module depends on. Both metrics are indicators of dependency: a module with high fan-in or fan-out has a higher potential for change propagation.
- **Instability Metric**
  - Measures the balance between the number of incoming and outgoing dependencies for a component. A component with high instability (a high number of outgoing dependencies) is more likely to propagate changes.

Cohesion Metrics

- Measures the degree to which the elements of a module or component are logically related. High cohesion indicates that a component or module is self-contained, reducing the risk of unintended consequences from changes.
- **Lack of Cohesion in Methods:**
  - Measures how related the methods within a class are to each other. A higher LCOM indicates poor cohesion, which can lead to unpredictable impacts when changes are made.
- **Cohesion Between Objects:**
  - Measures the degree to which different classes or modules interact and whether these interactions are necessary. Lower cohesion across objects increases the chance of higher impact when changes occur.

Dependency Metrics

- Measures how dependent different parts of the software are on each other. The more dependencies a component has, the greater the chance that changes will propagate across the system.
- **Data Dependency Metric**
  - Measures how much data is shared between components. Components with high data dependency will likely propagate changes to each other.
- **Control Dependency Metric**
  - Measures the control flow dependencies between components. A high control dependency indicates that a small change in one module might lead to significant changes in other parts of the system.
- **Module Dependency Metric**
  - Measures the number of modules dependent on each other. A high number indicates that the modules are tightly coupled, which increases the propagation risk when one module is changed.

Complexity Metrics

- Software complexity metrics can help quantify the difficulty of analyzing the impact of changes. More complex systems tend to have more dependencies and therefore, higher impact risks.
- **Cyclomatic Complexity**
  - Measures the number of linearly independent paths in a program's source code. A higher cyclomatic complexity indicates a more complex program with more possible interactions and potential impacts from changes.
- **Halstead Complexity**
  - Measures the complexity of software based on the number of operators and operands in the source code. Higher Halstead complexity often correlates with more challenging change management.
- **Essential Complexity**
  - A refinement of cyclomatic complexity that accounts for the essential control flow complexity (i.e., disregarding the code structure that doesn't contribute to decision-making logic).

Risk Metrics

- Risk-based metrics help assess the likelihood and severity of potential issues due to software changes.
- **Risk Factor**
  - Quantifies the risk of a change based on its complexity, the number of affected components, and the potential consequences of failure. A higher risk factor would indicate that a proposed change is more likely to cause problems.
- **Impact Risk Metric**
  - Evaluates the potential negative consequences of a change on the system's functionality or performance. Changes with high dependencies or complex interactions with other components will typically have higher impact risk.
- **Cost of Change**
  - The estimated effort and resources required to make the change and address any potential cascading impacts. This metric is useful for project planning and budgeting during re-engineering efforts.

Change Volume Metrics

- These metrics track the quantity of changes made to the system and can help identify areas of the system that are frequently modified, which could indicate high risk or technical debt.
- **Change Density**
  - Measures the number of changes (code modifications, updates, etc.) per module or class. Higher change density often means a higher likelihood of propagating changes.
- **Change Frequency**
  - Tracks how often a particular component or module is changed. Components that are changed frequently are more likely to experience cascading effects when modified.
- **Modification Propagation Ratio**
  - Tracks how many changes propagate from a single modification to other modules. A high ratio indicates that the module is involved in many interdependencies.

Traceability Metrics

- These metrics evaluate the ability to trace the relationships between software artifacts (requirements, code, tests, etc.) and understand how a change in one artifact might impact another.
- **Traceability Matrix**
  - A table that maps changes in source code to corresponding requirements, test cases, and other artifacts. This helps assess the overall impact on the system.
- **Requirement Coverage**
  - Measures the extent to which changes are aligned with the original requirements and whether the impacts of a change have been fully understood.

Questions?